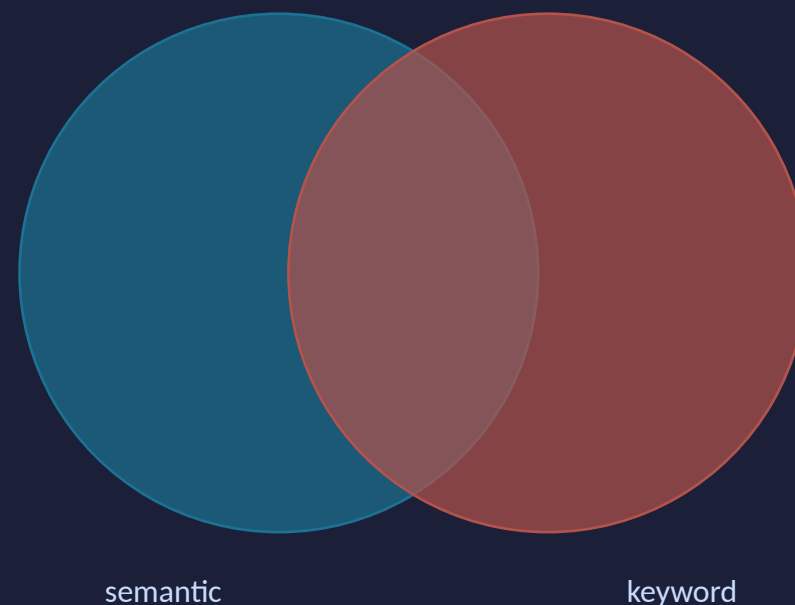


Hybrid Search

Our retriever cannot find the word “P100”.
That is not a bug — it is the mechanism working as designed.

Postgres • pgvector • LangChain • BM25 • RRF



What we said last time

Three ways to retrieve

1

Keyword (BM25)

Counts exact terms. Rare words weigh more.

2

Semantic (vectors)

Compares meaning. Words need not match.

3

Metadata filtering

Narrows the rows. Does not rank them.

Two ways semantic search fails

Similarity is not relevance

Opposites sit close together. “increase” and “decrease” are near-neighbours in embedding space.

Exact terms vanish

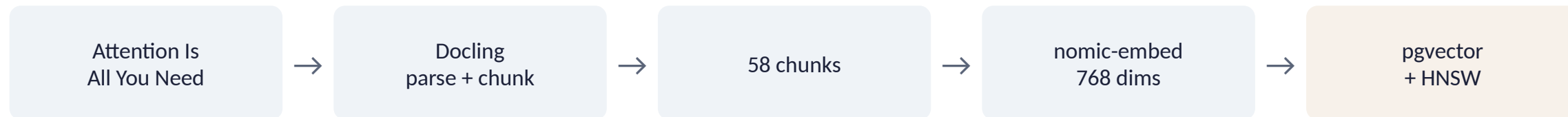
Error codes, model names, versions, figures. There is no meaning in ERR-409 to embed.

Today is entirely about the second one.

SETUP

The pipeline we are working with

INDEXING NOTEBOOK



QUERY NOTEBOOK



Everything here is semantic. One distance measure, one ranking. There is no keyword search anywhere in this pipeline — which is exactly the hole we are about to fall into.

Ask it for the authors' contact details

```
store.similarity_search_with_score("contact", k=3)
```

Rank	Chunk	Cosine distance
1	3.2 Attention	0.5309
2	3.2.1 Scaled Dot-Product Attention	0.5352
3	... avaswani@google.com ...	0.5570

The email is in the index. It came third.

Two section headings about attention beat the chunk that literally contains the address.

Nothing in the output said “I failed”

0.026

total spread across
all three results

1

Lower is better

These are distances, not similarities. The list ascends, so the best match is at the top.

2

All three are mid-0.5

Roughly a 0.47 cosine similarity. Weak. Nothing here is a confident match.

3

The tell is the flatness

When the top hits are separated by 0.026, the model is not discriminating. It is guessing politely.

Vector search always returns k results. It has no way to say “none of these are any good” — it returns its least-bad chunks with total confidence.

Why cosine cannot see “P100”

What cosine actually measures

The angle between two vectors — direction only, magnitude discarded.

That is deliberate: a two-line chunk and a two-paragraph chunk on the same topic compete fairly.

What “P100” is

A token with almost no semantic content. A letter and a number.

There is no direction in meaning-space for it to point in. Cosine has nothing to grip.

The same holds for every identifier in the paper

WMT 2014

BLEU

P100

$\epsilon_{ls} = 0.1$

$3.3 \cdot 10^{18}$

Adam

ERR-409

Cosine is not broken here. It is doing precisely what it was built to do — and that job does not include exact matching.

BM25: the opposite instrument

It understands nothing. It counts exact terms, weights rare ones more heavily, and adjusts for chunk length. That is the entire algorithm.

1

Term frequency

How often does this word appear in this chunk?

2

Inverse doc frequency

How rare is the word across all 58 chunks? Rare wins.

3

Length normalisation

A long chunk does not win just for being long.

Opposite blind spots

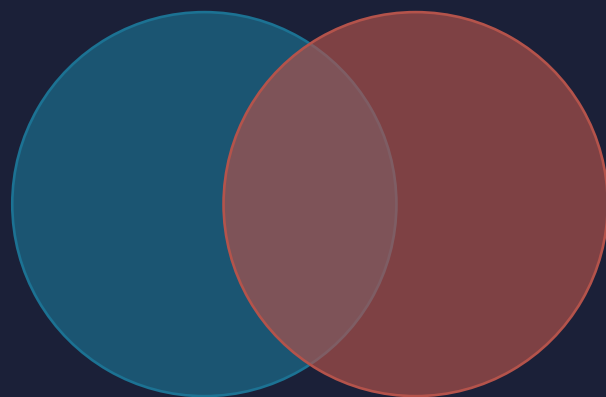
Cosine understands meaning and cannot match a string. BM25 matches strings and understands nothing. Neither is a better version of the other — which is why you fuse them rather than pick one.

Two queries. Opposite winners.

Query	Semantic	BM25	Why
P100	fails	finds it	Nothing to embed. Everything to match.
“how do you stop the decoder seeing future words”	finds it	fails	The chunk never uses those words. It says “masking”.

Neither retriever is wrong. Each is blind in the place the other sees.

Run all four combinations live. Watching the same pipeline win and then lose on two queries does more work than any diagram — and it sets up the only sensible conclusion: keep both, and merge the results.



So don't choose.

Reciprocal Rank Fusion

Reciprocal Rank Fusion

$$\text{score}(\text{doc}) = \sum 1 / (60 + \text{rank})$$

Sum over every result list the document appears in. The 60 is a damping constant — it stops rank 1 from dominating. You do not tune it.

Why this is the right merge

Scores are thrown away

Only position survives.

Nothing needs normalising

Cosine distance runs 0–2; BM25 is unbounded. Never compared.

Agreement is rewarded

A chunk both retrievers rank highly wins on the sum.

This is why RRF is the default in practice: it is the one fusion method that needs no calibration between two systems that share no units.

Ten lines of LangChain

```
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

bm25 = BM25Retriever.from_documents(chunks)    # a Python list
bm25.k = 5

hybrid = EnsembleRetriever(
    retrievers=[retriever, bm25],             # semantic, keyword
    weights=[0.5, 0.5],                       # fused by RRF
)

hybrid.invoke("P100")
```

It slots in at the seam

Everything downstream only knows “retriever”. The prompt, the chain and the model do not change at all.

Weights are a dial

50/50 is a starting point, not an answer. Domains heavy in identifiers deserve more weight on BM25.

Run the P100 query again. The chunk that ranked third now ranks first — that is the whole payoff.

Now the part the tutorials skip

BM25Retriever never touches your database.

You hand it a Python list. It builds a word index in memory and searches that. Your vectors live in Postgres; your keyword index lives in the notebook.

1

It dies with the kernel

Restart, and the index is gone. You reload and rebuild it every time.

2

It goes stale silently

New rows land in Postgres. BM25 has no idea they exist.

3

It duplicates your corpus

58 chunks in the database, 58 more in RAM. Now multiply by a real corpus.

Perfectly fine for this demo. Not an architecture. Say so before someone ships it.

Where the keyword index should live



Postgres full-text search

A tsvector column, a GIN index, ts_rank_cd. Both halves in one database, updated automatically.

Not true BM25 — same spirit, different formula. You fuse the ranks yourself.



ParadeDB (pg_search)

A Postgres extension that adds genuine BM25 scoring inside the database you already run.

An extension to install and operate. Fine if you control the instance.



Elasticsearch / OpenSearch

Real BM25 and hybrid retrieval out of the box. LangChain ships a store for it.

A second datastore to run, sync and pay for. Justify it before you add it.

pgvector does vectors. It does not do keywords. That gap is the same one we found in Spring AI — LangChain being further along still leaves the work to you.

Hybrid search does not fix these

1

Garbage in the chunk

Docling read the author block as a table and serialised it as “Ashish Vaswani * Google Brain avaswani@google.com, 1 = Noam Shazeer...”. No retrieval strategy recovers from a bad parse.

2

Confidently wrong citations

Every answer cites “Source: Page 0”. The page number was read from the wrong metadata key. A wrong citation is worse than none at all.

3

Five chunks saying one thing

That is a diversity problem, not a vocabulary problem. MMR is the fix — it penalises candidates similar to what it already picked.

Different failures need different fixes. Reaching for hybrid search on a parsing bug wastes a day.

How would you know it worked?

Right now we judge retrieval by reading it. That does not scale, and it does not survive a config change.

The minimum that is useful

A small set of questions with the chunk you know is correct.

Then two numbers:

- did the right chunk come back?
- how high did it rank?

That is enough to claim anything

Ten questions and a hit-rate lets you say “hybrid beat semantic by this much” instead of “it feels better”.

Session Eight: evaluation properly — RAGAS, faithfulness, LLM-as-judge, and why the last one is harder than it looks.

Four things to leave with

1

Vector search never says “I don't know”

It returns k results regardless. Read the spread between them, not the top score.

2

Embeddings cannot match strings

Identifiers, codes and figures have no meaning to compress. That is by design.

3

RRF fuses ranks, not scores

Which is the only reason two systems with incompatible units can be merged at all.

4

Know which failure you have

Vocabulary, diversity, parsing and citation are four different bugs with four different fixes.